

Task01 – Pozíciókezelő**Cél**

Készítsen egy egyszerű ASP.NET Core MVC alkalmazást, amely lehetővé teszi munkakörök tárolását és megjelenítését. A feladat célja az alapvető MVC struktúra gyakorlása, valamint annak megértése, hogyan injektálható egy repository a controllerbe az IoC konténeren keresztül.

Követelmények

Hozzon létre egy ASP.NET Core Empty projektet, majd kézzel bővítse MVC alkalmazássá.

Az alkalmazásnak tartalmaznia kell az alábbi mappákat:

- Models
- Data
- Controllers
- Views
- wwwroot

Funkcionális követelmények

Az alkalmazás tegye lehetővé a felhasználó számára:

- kezdőoldal megnyitását
- új munkakör beírását
- az új munkakör beküldését
- az összes eltárolt munkakör megjelenítését

Példák munkakörökre:

- developer
- tester
- manager
- analyst

Strukturális követelmények**1. Program.cs**

Konfigurálja az alkalmazást úgy, hogy:

- az MVC controllerek és view-k engedélyezve legyenek
- egy repository regisztrálva legyen az IoC konténerben
- az útválasztás engedélyezve legyen az alapértelmezett route mintával: `{controller}/{action}/{id?}`

A repository-t interfészen keresztül kell regisztrálni, nem közvetlenül a konkrét típussal.

2. Model

Hozzon létre egy modell osztályt, amely egy pozíciót reprezentál.

A modell legalább az alábbi tartalmazza:

- a pozíció neve

Egyszerű sztring alapú belső tárolás elfogadható, de az alkalmazásnak ettől függetlenül tartalmaznia kell megfelelő modell osztályt.

3. Repository

A Data mappában:

- hozzon létre egy interfészt a pozíciók kezeléséhez
- hozzon létre egy repository osztályt, amely megvalósítja az interfészt

A repository-nak biztosítania kell az alábbi műveleteket:

- új pozíció hozzáadása
- az összes eltárolt pozíció visszaadása

Használjon memóriában történő tárolást. Adatbázis nem szükséges.

4. Controller

Hozzon létre egy PositionsController-t.

A controller konstruktor injektálással kapja meg a repository-t annak interfészén keresztül.

A controller legalább az alábbi actionöket tartalmazza:

- Index
- Add
- List

Elvárt működés:

- az Index megjeleníti a beviteli űrlapot
- az Add fogadja a beírt pozíciót és eltárolja azt a repository segítségével
- hozzáadás után a felhasználó átirányításra kerül a lista oldalra
- a List megjeleníti az eltárolt pozíciókat

5. View-k

Hozza létre a szükséges Razor view-kat.

Minimum:

- egy oldal űrlappal új pozíció megadásához
- egy oldal az aktuális pozíciólista megjelenítéséhez

A lista legyen jól olvasható és áttekinthető formában megjelenítve.

UI követelmények

Hozzon létre egy egyszerű külső CSS fájlt a wwwroot/css mappában.

Alkalmazzon alap stílusokat az alábbiakhoz:

- oldalcím
- űrlap
- beviteli mező
- gombok
- pozíciólista

Task02 – Termékkatalógus kategória szerinti szűréssel

Cél

Készítsen egy ASP.NET Core MVC alkalmazást termékek kezelésére. Az alkalmazás mutassa be, hogy a controller csak absztrakciótól függ, miközben a repository végzi a tényleges adatkezelést. A feladat egyszerű szűrési logikát is bevezet.

Követelmények

Hozzon létre egy ASP.NET Core MVC alkalmazást Empty projektből.

A projektnek tartalmaznia kell az alábbi mappákat:

- Models
- Data
- Controllers
- Views
- wwwroot

Funkcionális követelmények

Az alkalmazás tegye lehetővé a felhasználó számára:

- új termék hozzáadását
- a termék kategóriájának megadását
- az összes termék megjelenítését
- csak a kiválasztott kategóriába tartozó termékek megjelenítését

Példa kategóriák:

- Electronics
- Food
- Office
- Books

Strukturális követelmények

1. Model

Hozzon létre egy Product modellt.

A modell legalább az alábbiakat tartalmazza:

- termék neve
- kategória
- ár

2. Repository interfész és implementáció

A Data mappában:

- hozzon létre egy interfészt a termékekkel kapcsolatos műveletekhez
- hozzon létre egy repository osztályt, amely megvalósítja az interfészt

A repository legalább az alábbi műveleteket támogassa:

- termék hozzáadása
- összes termék visszaadása
- adott kategóriába tartozó termékek visszaadása

Az adatokat memóriában tárolja.

3. IoC konfiguráció

A Program.cs fájlban:

- engedélyezze az MVC-t
- regisztrálja a repository-t annak interfészén keresztül
- konfigurálja az alapértelmezett MVC útválasztást

A controller konstruktor injektálással kapja meg a függőséget.

4. Controller

Hozzon létre egy ProductsController-t.

Legalább az alábbi actionöket tartalmazza:

- Index
- Add
- List
- ByCategory

Elvárt működés:

- az Index megjelenít egy űrlapot a termékadatok beviteléhez
- az Add eltárol egy új terméket a repository használatával
- a List megjeleníti az összes eltárolt terméket

- a ByCategory csak a kiválasztott kategóriába tartozó termékeket jeleníti meg

5. View-k

Hozzon létre Razor view-kat a controller actionökhöz.

A felhasználói felület tartalmazza:

- szöveges beviteli mező a termék nevéhez
- szöveges beviteli mező vagy legördülő lista a kategóriához
- numerikus beviteli mező az árhoz
- küldés gomb
- linkek vagy vezérlők kategória alapú szűréshez
- formázott táblázat vagy lista a termékek megjelenítéséhez

További követelmények

A controller nem hozhatja létre a repository-t new használatával.

A controller nem tárolhat termékeket lokális List<Product> mezőben.

Minden adatkezelési műveletnek a repository-n keresztül kell történnie.

Task03 – Hallgatói nyilvántartó két repository-val

Cél

Készítsen egy összetettebb ASP.NET Core MVC alkalmazást, amely több repository-t használ dependency injection segítségével. A cél annak bemutatása, hogy egy controller több szolgáltatással is tud együttműködni, és minden szolgáltatásnak saját felelősségi köre van.

Szituáció

Az alkalmazás hallgatókat és osztályokat kezel.

Egy hallgató egy osztályhoz tartozik, és a felhasználónak lehetősége kell legyen:

- új hallgatók hozzáadására
- új osztályok hozzáadására
- az összes osztály megtekintésére
- az összes hallgató megtekintésére
- egy kiválasztott osztályhoz tartozó hallgatók megjelenítésére

Követelmények

Hozza létre az alkalmazást ASP.NET Core Empty projektből, és kézzel építse fel az MVC struktúrát.

Az alkalmazásnak tartalmaznia kell:

- Models
- Data
- Controllers
- Views
- wwwroot

Strukturális követelmények

1. Modellek

Hozzon létre legalább két modell osztályt:

- Student
- SchoolClass (vagy igény szerint Classroom)

Javasolt tulajdonságok:

Student

- név
- életkor
- osztály neve vagy osztály azonosítója

SchoolClass

- osztály neve
- teremszám vagy rövid leírás

2. Repository-k

A Data mappában hozzon létre:

- egy interfészt és implementációt a hallgatók kezelésére
- egy interfészt és implementációt az osztályok kezelésére

Példák:

hallgató repository:

- hallgató hozzáadása
- összes hallgató listázása
- hallgatók listázása osztály szerint

osztály repository:

- osztály hozzáadása
- összes osztály listázása

Használjon memóriában történő tárolást.

3. Program.cs

Konfigurálja az alkalmazást úgy, hogy:

- az MVC engedélyezve legyen
- mindkét repository regisztrálva legyen az IoC konténerben
- az útválasztás a szabványos MVC route szerint legyen konfigurálva

4. Controllerek

Hozzon létre legalább két controllert:

- StudentsController
- ClassesController

StudentsController

A controller konstruktor injektálással kapja meg a hallgató repository-t. Szükség esetén az osztály repository-t is megkaphatja az osztályválasztások megjelenítéséhez.

Javasolt actionök:

- Index
- Add
- List
- ByClass

ClassesController

A controller konstruktor injektálással kapja meg az osztály repository-t.

Javasolt actionök:

- Index
- Add
- List

5. View-k

Hozzon létre olyan view-kat, amelyek könnyen használhatók teszik a funkcionalitást.

A felhasználói felület támogassa:

- osztály hozzáadását
- az összes osztály listázását
- hallgató hozzáadását
- osztály hozzárendelését a hallgatóhoz
- az összes hallgató listázását
- egy kiválasztott osztály hallgatóinak listázását

További elvárások

A hallgatók fordítsanak figyelmet az alábbi tervezési szabályokra:

- az adatokat nem szabad közvetlenül a controllerben tárolni
- a repository-khoz interfészeken keresztül kell hozzáférni
- az IoC konténernek kell létrehoznia és biztosítania a repository példányokat
- a controllereknek a kérések koordinálását kell végezniük, nem az adattárolási logikát megvalósítaniuk

Opcionális bővítés

Adjon hozzá navigációs linkeket az osztályokkal és hallgatókkal kapcsolatos oldalak között.

Lehetőség szerint készítsen közös layoutot és külső CSS fájlt az egységesebb megjelenés érdekében.